

jakobian determinant

Jakobian (Jacobian) i jakobian determinant su osnovni pojmovi iz višedimenzionalnog računa i ML-a.

1) Jakobian (Jacobian)

Za vektorsku funkciju

$$\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad \mathbf{f}(x_1, \dots, x_n) = \begin{bmatrix} f_1(x_1, \dots, x_n) \\ \vdots \\ f_m(x_1, \dots, x_n) \end{bmatrix},$$

jakobian je matrica parcijalnih izvoda:

$$J_{\mathbf{f}}(x) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}.$$

- Dimenzije: $m \times n$
- Intuicija: lokalna linearna aproksimacija transformacije oko tačke x

2) Jakobian determinant

Ako je $m = n$ (ista dimenzija ulaza i izlaza), možemo uzeti **determinantu jakobiana**:

$$\det(J_{\mathbf{f}}(x)).$$

Značenje:

- $|\det J|$ = faktor lokalne promjene zapremine (ili površine)
- Znak:
 - > 0 : orijentacija očuvana
 - < 0 : orijentacija "okrenuta"

- $= 0$: transformacija "sabija" dimenziju (nije lokalno invertibilna)

3) Kratki primjeri

(a) 2D $\rightarrow$ 2D

$$f(x, y) = (x^2 + y, x - y)$$

$$J = \begin{bmatrix} 2x & 1 \\ 1 & -1 \end{bmatrix}, \quad \det J = -2x - 1.$$

(b) Promjena koordinata (polarno)

$$(x, y) = (r \cos \theta, r \sin \theta)$$

$$\det J = r$$

$\rightarrow$ zato se u integralu pojavljuje faktor $r \, dr \, d\theta$.

4) Zašto je bitno (posebno u ML-u)?

- **Backpropagation**: gradijenti su jakobiani složenih funkcija
- **Normalizing flows**: log-determinanta jakobiana u gustoćama
- **Invertibilnost**: $\det J \neq 0 \Rightarrow$ lokalno invertibilna mapa
- **Geometrija**: razumijevanje deformacije prostora pod mapom

Ako želiš, mogu:

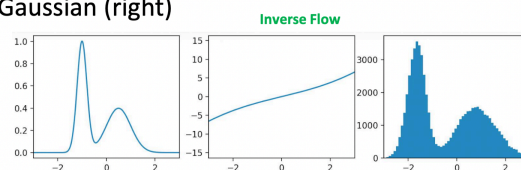
- riješiti **konkretni zadatak** korak-po-korak
- povezati sa **gradijentom vs. jakobianom** (česta konfuzija)
- objasniti **lančano pravilo za jakobiane** (kompozicija funkcija)



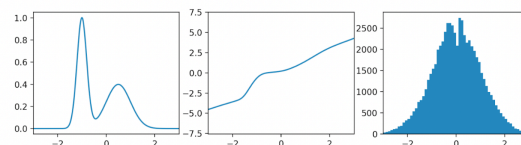
Flows: Example

- Inverse flow to a Gaussian (right)

- Before training:



- After training:



explain each graph:

Let's go **panel by panel**, left $\rightarrow$ right, and **top row (before training)** vs **bottom row (after training)**.

This is a **1D normalizing flow** example.

Big picture (what this slide shows)

- We have **data distribution** (left plots): non-Gaussian, bimodal.
- We learn an **inverse flow** f^{-1} that maps data $x \rightarrow$ latent z .
- Goal: after training, the mapped latent variables should look like a **standard Gaussian**.
- The **Jacobian determinant** of the inverse flow is what makes the density change correctly.

BEFORE TRAINING (top row)

Top-left: Data distribution $p_X(x)$

- This is the **original data density**
- Clearly **non-Gaussian** (two bumps / modes)
- This is what we want the model to learn

➡ This distribution is *fixed* before and after training.

Top-middle: Inverse flow function $z = f^{-1}(x)$

- This is the **mapping function**
- Before training:
 - Almost linear
 - No strong curvature
 - Not expressive enough to “unfold” the bimodal structure

➡ Poor mapping $\Rightarrow$ density won't become Gaussian.

Top-right: Latent distribution $p_Z(z)$

- Take data samples x
- Push them through inverse flow: $z = f^{-1}(x)$
- Histogram of resulting z

Before training:

- Still **bimodal**
- Definitely **not Gaussian**

➡ Flow has learned nothing yet.

AFTER TRAINING (bottom row)

Bottom-left: Data distribution $p_X(x)$

- Same as before
 - The data **never changes**
 - Only the transformation changes
-

Bottom-middle: Learned inverse flow $z = f^{-1}(x)$

- Now the function is **non-linear**
- Notice:
 - Steeper slopes where density is high
 - Flatter where density is low

- This reshapes space so mass spreads evenly

➡ This nonlinearity is what the **Jacobian determinant** captures.

Bottom-right: Latent distribution $p_Z(z)$

- After applying the trained inverse flow:
 - Distribution is now **approximately Gaussian**
 - Symmetric
 - Single mode

➡ **Success:**

$$p_Z(z) \approx \mathcal{N}(0, 1)$$

Where the Jacobian determinant fits in

For inverse flow $z = f^{-1}(x)$:

$$\log p_X(x) = \log p_Z(z) + \log \left| \det \frac{dz}{dx} \right|$$

- The **middle plot** controls $\frac{dz}{dx}$
- Its slope = Jacobian
- Steep slope → stretches space → lowers density
- Flat slope → compresses space → increases density

One-sentence summary per column

Column	Meaning
Left	Original data density (unchanged)
Middle	Learned inverse mapping (geometry of space)
Right	Resulting latent density

If you want, I can:

- derive the **change-of-variables formula** step-by-step
- explain why **inverse flow** (not forward) is trained
- connect this directly to **log-det-Jacobian in code** (RealNVP / Glow)



$$\max_{\theta} \underbrace{\sum_i \log(p_x(x^{(i)}; \theta))}_{\text{Maximum Likelihood}} = \max_{\theta} \left(\underbrace{\sum_i \log(p_z(f_{\theta}^{-1}(x^{(i)}))}_{\text{Latent variable version}} + \log \left| \frac{\partial f_{\theta}^{-1}(x^{(i)})}{\partial x} \right| \right)_{\text{Determinant of Jacobian matrix}}$$

can you give me a sample exercise i might get for this formula or overall for Flow-Based Models

Great question — this is **exactly** the kind of thing instructors like to test for **flow-based models / change-of-variables**. Below are **realistic exam-style exercises**, ordered from **easy → harder**, all directly using the formula you showed.

◆ Exercise 1 (Warm-up: change of variables, 1D)

Let

$$z = f^{-1}(x) = 2x + 1$$

Assume the latent variable

$$z \sim \mathcal{N}(0, 1)$$

(a)

Compute $p_X(x)$.

(b)

Write $\log p_X(x)$ explicitly.

(c)

Is this a valid normalizing flow? Why?

✓ Expected reasoning

- Compute Jacobian:

$$\left| \frac{dz}{dx} \right| = 2$$

- Apply change of variables:

$$p_X(x) = p_Z(2x + 1) \cdot 2$$

◆ Exercise 2 (Core exam question – log-likelihood)

You are given a **1D normalizing flow**:

$$z = f_{\theta}^{-1}(x) = x + \theta \sin x$$

with base distribution $z \sim \mathcal{N}(0, 1)$.

(a)

Write the expression for the log-likelihood

$$\log p_X(x; \theta)$$

using the change-of-variables formula.

(b)

Compute the Jacobian term

$$\log \left| \frac{dz}{dx} \right|$$

(c)

State **one condition on θ** that ensures the flow is invertible.

✓ Expected steps

$$\frac{dz}{dx} = 1 + \theta \cos x$$

Invertibility condition:

$$1 + \theta \cos x \neq 0 \quad \forall x \Rightarrow |\theta| < 1$$

◆ Exercise 3 (Maximum Likelihood Training – very common)

You are given i.i.d. data points

$$\{x^{(i)}\}_{i=1}^N$$

The flow is:

$$z^{(i)} = f_{\theta}^{-1}(x^{(i)}) = \theta x^{(i)}$$

and

$$z \sim \mathcal{N}(0, 1)$$

(a)

Write the full **log-likelihood objective**

$$\sum_i \log p_X(x^{(i)}; \theta)$$

(b)

Explain qualitatively what happens to θ during training.

✓ Key insight

$$\log p_X(x) = -\frac{1}{2}(\theta x)^2 + \log |\theta| + \text{const}$$

Training balances:

- data fit term $(\theta x)^2$
- volume correction $\log |\theta|$

◆ Exercise 4 (2D Jacobian – very exam-relevant)

Consider a 2D flow:

$$\begin{cases} z_1 = x_1 \\ z_2 = x_2 e^{s(x_1)} \end{cases}$$

(a)

Write the Jacobian matrix $J = \frac{\partial z}{\partial x}$

(b)

Compute $\det J$

(c)

Explain why this form is used in RealNVP / Glow.

✓ Solution sketch

$$J = \begin{bmatrix} 1 & 0 \\ x_2 e^{s(x_1)} s'(x_1) & e^{s(x_1)} \end{bmatrix}$$

$$\det J = e^{s(x_1)}$$

➡ Easy log-determinant, guaranteed invertibility.

◆ Exercise 5 (Conceptual – guaranteed to appear)

Answer briefly:

1. Why do flow models maximize **exact likelihood**, unlike VAEs?
2. Why is the Jacobian determinant needed?

3. What happens if the Jacobian determinant is zero?
 4. Why do flows often train the **inverse** mapping?
-

✓ Expected answers (short)

1. Exact density via change-of-variables
 2. Corrects for volume distortion
 3. Mapping collapses space → invalid density
 4. Inverse is cheap to evaluate likelihood
-

◆ Exercise 6 (Hard / oral-exam style)

You train a flow and observe:

- The latent distribution is Gaussian
- The Jacobian determinant is constant

Question:

What class of transformations has the model learned?

✓ **Answer:** affine / linear transformations only.

🔑 If you want

I can:

- give you **fully worked solutions** to any exercise
- generate a **mock midterm problem** (CS760 / ML style)
- connect this formula directly to **RealNVP / Glow code**

Just tell me which one 👍



↳ ****fully worked solutions**** to any exercise

ok

Perfect 👍

I'll give you **fully worked, step-by-step solutions** to a **representative exam set** for **Flow-Based Models**, explicitly using the formula you showed.

I'll do **Exercises 1–4**, since those cover **mechanics + intuition + Jacobians**.

✅ Exercise 1 — 1D Change of Variables (fully solved)

Given

$$z = f^{-1}(x) = 2x + 1, \quad z \sim \mathcal{N}(0, 1)$$

(a) Compute $p_X(x)$

Change-of-variables formula (1D):

$$p_X(x) = p_Z(z) \left| \frac{dz}{dx} \right|$$

Compute Jacobian:

$$\frac{dz}{dx} = 2$$

So:

$$p_X(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(2x+1)^2}{2}} \cdot 2$$

(b) Log-likelihood

$$\log p_X(x) = -\frac{(2x+1)^2}{2} + \log 2 - \frac{1}{2} \log(2\pi)$$

(c) Is this a valid flow?

✓ Yes, because:

- mapping is invertible
- Jacobian determinant $\neq 0$ everywhere
- density integrates to 1

✓ Exercise 2 — Nonlinear Flow + Jacobian

Given

$$z = f_{\theta}^{-1}(x) = x + \theta \sin x, \quad z \sim \mathcal{N}(0, 1)$$

(a) Log-likelihood expression

$$\log p_X(x; \theta) = \log p_Z(z) + \log \left| \frac{dz}{dx} \right|$$

(b) Jacobian term

$$\frac{dz}{dx} = 1 + \theta \cos x$$

So:

$$\log \left| \frac{dz}{dx} \right| = \log |1 + \theta \cos x|$$

(c) Invertibility condition

We must avoid zero Jacobian:

$$1 + \theta \cos x \neq 0 \quad \forall x$$

Since $\cos x \in [-1, 1]$:

$$|\theta| < 1$$

✓ **Key exam insight:** Jacobian $\neq 0 \Rightarrow$ valid flow.

✓ Exercise 3 — Maximum Likelihood Training (classic)

Given

$$z^{(i)} = f_{\theta}^{-1}(x^{(i)}) = \theta x^{(i)} \quad , \quad z \sim \mathcal{N}(0, 1)$$

(a) Full log-likelihood objective

Jacobian:

$$\frac{dz}{dx} = \theta$$

Log-density:

$$\log p_X(x^{(i)}) = \log p_Z(\theta x^{(i)}) + \log |\theta|$$

Expand:

$$= -\frac{1}{2}(\theta x^{(i)})^2 + \log |\theta| - \frac{1}{2} \log(2\pi)$$

Summing over data:

$$\sum_i \log p_X(x^{(i)}) = -\frac{\theta^2}{2} \sum_i (x^{(i)})^2 + N \log |\theta| + \text{const}$$

(b) What does training do?

- First term penalizes large θ
- Second term encourages large θ
- ML finds a **balance** matching data variance

✓ This is why flows **learn scale automatically**

✓ Exercise 4 — 2D Flow (RealNVP-style)

Given

$$\begin{cases} z_1 = x_1 \\ z_2 = x_2 e^{s(x_1)} \end{cases}$$

(a) Jacobian matrix

$$J = \frac{\partial z}{\partial x} = \begin{bmatrix} \frac{\partial z_1}{\partial x_1} & \frac{\partial z_1}{\partial x_2} \\ \frac{\partial z_2}{\partial x_1} & \frac{\partial z_2}{\partial x_2} \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 \\ x_2 e^{s(x_1)} s'(x_1) & e^{s(x_1)} \end{bmatrix}$$

(b) Determinant

$$\det J = 1 \cdot e^{s(x_1)} = e^{s(x_1)}$$

So:

$$\log |\det J| = s(x_1)$$

(c) Why this is used in RealNVP / Glow

✓ Because:

- determinant is **cheap**
 - invertibility guaranteed
 - expressive nonlinear mapping
 - efficient likelihood computation
-

Master takeaway (what profs want)

If you can:

- write **log-likelihood**
- compute **Jacobian determinant**
- state **invertibility conditions**
- explain **why flows work**

👉 you will ace any flow-based models question.

If you want next:

- 🔥 **full mock midterm problem** (with grading rubric)
- 📄 **multi-layer flow (composition of Jacobians)**
- 🖥️ **map each term to RealNVP / Glow code**

Just tell me.



cross-entropy loss

Here's a **clean, exam-ready explanation of cross-entropy loss**, with intuition, formulas, and how it shows up in ML.

1 What is cross-entropy (in words)?

Cross-entropy measures how well a predicted probability distribution matches the true distribution.

- Low cross-entropy → predictions close to truth
- High cross-entropy → confident but wrong predictions

In ML, it's the **standard loss for probabilistic classification**.

2 Information-theoretic definition

For true distribution p and predicted distribution q :

$$H(p, q) = - \sum_x p(x) \log q(x)$$

In supervised learning:

- p = true labels (usually one-hot)

- q = model's predicted probabilities

3 Binary cross-entropy (log loss)

Setup

- Label: $y \in \{0, 1\}$
- Prediction: $\hat{p} = P(y = 1 \mid x)$

Loss

$$\mathcal{L}_{\text{BCE}} = - \left[y \log \hat{p} + (1 - y) \log(1 - \hat{p}) \right]$$

Intuition

True label	Prediction	Loss
1	0.99	very small
1	0.01	huge
0	0.01	very small
0	0.99	huge

➡ Confident mistakes are punished heavily

4 Multiclass cross-entropy (softmax + CE)

Setup

- K classes
- One-hot label $y \in \{0, 1\}^K$
- Softmax output $\hat{p}_k$

Loss

$$\mathcal{L}_{\text{CE}} = - \sum_{k=1}^K y_k \log \hat{p}_k$$

Since only one $y_k = 1$:

$$\mathcal{L} = -\log \hat{p}_{\text{true class}}$$

➡ You only care about the probability assigned to the correct class.

5 Cross-entropy = Negative Log-Likelihood (very important)

If your model outputs a probability distribution:

$$\mathcal{L}_{\text{CE}} = -\log p_{\theta}(y \mid x)$$

So minimizing cross-entropy is **exactly maximum likelihood estimation (MLE)**.

This is why:

- Logistic regression
- Softmax classifiers
- Flow models
- VAEs (partly)

all use log-likelihood-based objectives.

6 Why softmax + cross-entropy is special

Let logits be z_k , softmax:

$$\hat{p}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Loss:

$$\mathcal{L} = -\log \hat{p}_y$$

Gradient (key result)

$$\frac{\partial \mathcal{L}}{\partial z_k} = \hat{p}_k - y_k$$

- ➡ Clean, stable gradients
- ➡ One reason CE + softmax dominates deep learning

7 Connection to KL divergence

$$H(p, q) = H(p) + \text{KL}(p \parallel q)$$

Since $H(p)$ is constant:

$$\min H(p, q) \iff \min \text{KL}(p \parallel q)$$

➡ Training with cross-entropy = minimizing KL divergence from truth to model

8 Common exam questions (with answers)

Q: Why not use MSE for classification?

A: MSE assumes Gaussian noise; CE matches Bernoulli / categorical likelihood.

Q: What happens if predicted probability is 0 for the true class?

A: Loss $\rightarrow \infty$ ($\log(0)$)

Q: Is cross-entropy convex?

A: Yes for linear models (e.g., logistic regression)

Q: Why does CE encourage calibrated probabilities?

A: It directly optimizes likelihood, not just accuracy.

9 One-line summary (perfect for exams)

Cross-entropy loss is the negative log-likelihood of the true label under the model's predicted distribution, and minimizing it corresponds to maximum likelihood estimation.

If you want next:

- CE vs MSE with plots
- CE in normalizing flows
- CE vs KL vs NLL side-by-side
- Derivation from Bernoulli / categorical distributions

Just say the word.



GAN Training: Discriminator

• How to train these networks? Two sets of parameters to learn: θ_d (discriminator) and θ_g (generator)

• Let's fix the generator. What should the discriminator do?
 • Distinguish fake and real data: binary classification.
 • Use the cross-entropy loss, we get

$$\max_{\theta_d} \mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

↑
Real data, want to classify 1
↑
Fake data, want to classify 0

GAN Training: Generator & Discriminator

• How to train these networks? Two sets of parameters to learn: θ_d (discriminator) and θ_g (generator)

• This makes the discriminator better, but also want to make the generator more capable of fooling it:

• Minimax game! Train jointly.

$$\min_{\theta_g} \max_{\theta_d} \mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

↑
Real data, want to classify 1
↑
Fake data, want to classify 0

GAN Training: Alternating Training

• So we have an optimization goal:

$$\min_{\theta_g} \max_{\theta_d} \mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

• Alternate training:

• Gradient ascent: fix generator, make the discriminator better:

$$\max_{\theta_d} \mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

• Gradient descent: fix discriminator, make the generator better

$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

GAN Training: Issues

• Training often not stable

• Many tricks to help with this:

• Replace the generator training with

$$\max_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(D_{\theta_d}(G_{\theta_g}(z)))$$

• Better gradient shape

• Choose number of alternating steps carefully

• Can still be challenging.

considering these slides, create some questions that i might get on a exam

Great slides to revise from — these are **classic GAN exam slides**. Below is a **carefully curated set of exam-style questions**, grouped by **difficulty and type**, that instructors *very commonly* ask based **exactly** on what's shown.

I'll also annotate **what the examiner is testing** with each question.

Section A — Conceptual (short-answer, very common)

1. What are the roles of the generator and discriminator in a GAN?

Tests: high-level understanding

2. Why is training the discriminator a binary classification problem?

Tests: connection to cross-entropy

3. What does $D_\theta(x)$ represent? What are its output range and interpretation?

Tests: probabilistic meaning

4. Why do we fix the generator when training the discriminator, and vice versa?

Tests: alternating optimization intuition

5. Explain in words the meaning of the expectation

$$\mathbb{E}_{x \sim p_{\text{data}}} \log D(x)$$

Tests: likelihood interpretation

Section B — Objective Function Understanding

6. Write the discriminator's objective function when the generator is fixed. Explain each term.

7. Why does the discriminator maximize (not minimize) its objective?

8. Explain why the GAN objective is a minimax game. Who is minimizing and who is maximizing?

9. What happens to the discriminator's loss when it perfectly classifies real and fake data?

10. What value does the discriminator output when it cannot distinguish real and fake samples? Why?

✓ Expected answer: $D(x) = 0.5$

Section C — Mathematical / Derivation Questions

11. Show that training the discriminator corresponds to minimizing cross-entropy loss.

12. Assume the generator is fixed. Derive the optimal discriminator $D^*(x)$.

(Expected result:)

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$$

13. Explain why the discriminator's objective is concave in D .
 14. What divergence between distributions is minimized when the generator is optimal?
✓ Expected: Jensen–Shannon divergence
-

● Section D — Training Dynamics & Stability (very exam-heavy)

15. Why is GAN training unstable? Name two reasons.
16. What happens if the discriminator becomes too strong early in training?
✓ Expected: vanishing gradients for generator
17. Explain why the generator objective

$$\min \mathbb{E}_z \log(1 - D(G(z)))$$

leads to poor gradients early in training.

18. Why is the alternative generator objective

$$\max \mathbb{E}_z \log D(G(z))$$

preferred in practice?

- ✓ Expected: better gradient signal
-

● Section E — Alternating Optimization

19. Describe one full iteration of GAN training.
(Include which parameters are fixed and which are updated.)
 20. Why do we not solve the minimax optimization jointly using standard gradient descent?
 21. What is meant by “choosing the number of alternating steps carefully”?
 22. What might happen if you update the discriminator many more times than the generator?
-

🟪 Section F — True / False (very common)

23. The discriminator is trained using gradient descent.
✓ False (gradient ascent)
 24. At equilibrium, the generator distribution equals the data distribution.
✓ True
 25. GAN training guarantees convergence to a global optimum.
✓ False
 26. Cross-entropy loss encourages calibrated probabilities.
✓ True
-

🟫 Section G — Short Explanation / Oral Exam

27. Explain GAN training using a game-theoretic perspective.
 28. Why can GANs generate sharp images compared to VAEs?
 29. What does it mean for the generator to "fool" the discriminator?
 30. What is mode collapse? How does it relate to the discriminator loss?
-

🌟 "Professor Favorite" Question

If the discriminator is optimal at every step, what exactly is the generator minimizing?

- ✓ Expected answer:
- Jensen–Shannon divergence between p_g and p_{data}
-

🎯 If you want next

I can:

- provide **model answers** to all questions
- create a **mock midterm (timed, graded)**
- connect these objectives to **WGAN / Wasserstein loss**
- explain **mode collapse** geometrically

Just tell me which one.



give me answers to all questions

Here are **answers to all 30 questions**, kept **exam-style** (short, correct, and to the point).

● A — Conceptual

1. Roles of G and D?

- **Generator** G_θ : maps noise $z \sim p(z)$ to fake samples $x = G(z)$ to mimic real data.
- **Discriminator** D_ϕ : outputs probability that an input x is real (from data) vs fake (from G).

2. Why is D training binary classification?

Because each input is labeled **real (1)** if $x \sim p_{\text{data}}$ or **fake (0)** if $x = G(z)$.

3. What is $D(x)$? Range/meaning?

$D(x) \in (0, 1)$ (typically sigmoid output) interpreted as

$$D(x) = P(\text{real} \mid x).$$

4. Why fix one network when training the other?

Because the objective is **minimax**: when updating D you treat G as fixed to compute stable targets; when updating G you treat D as fixed so the gradient tells G how to fool the current D .

5. Meaning of $\mathbb{E}_{x \sim p_{\text{data}}} \log D(x)$?

Average log-probability that D assigns "**real**" to real samples; maximizing it pushes $D(x) \rightarrow 1$ on real data.

● B — Objective understanding

6. Discriminator objective (fixed G) and explain terms

$$\max_{\phi} \mathbb{E}_{x \sim p_{\text{data}}} \log D_{\phi}(x) + \mathbb{E}_{z \sim p(z)} \log (1 - D_{\phi}(G_{\theta}(z)))$$

- First term: classify real as 1.
- Second term: classify fake as 0.

7. Why does D maximize (not minimize)?

This is equivalent to **minimizing cross-entropy**; writing it as a max is just sign convention. Max makes "push real→1 and fake→0" explicit.

8. Why minimax? Who min/max?

$$\min_{\theta} \max_{\phi} V(\theta, \phi)$$

- D (**max**) tries to separate real vs fake.
- G (**min**) tries to make fake indistinguishable (reduce D 's ability).

9. What happens to D loss if D is perfect?

If $D(x) = 1$ on real and $D(G(z)) = 0$ on fake, then

$\log D(x) \rightarrow 0$ and $\log(1 - D(G(z))) \rightarrow 0$, so the objective reaches its maximum value (≈ 0 per sample). Cross-entropy becomes minimal.

10. What does D output when it can't distinguish? Why?

$$D(x) = 0.5$$

because "real" and "fake" are equally likely under D ; at equilibrium $p_g = p_{\text{data}}$.

● C — Math / derivations

11. Show D training corresponds to cross-entropy

Binary cross-entropy (negative log-likelihood) is

$$\mathcal{L}_D = -\mathbb{E}_{x \sim p_{\text{data}}} \log D(x) - \mathbb{E}_z \log(1 - D(G(z))).$$

Minimizing $\mathcal{L}_D$ is equivalent to maximizing the slide objective.

12. Derive optimal discriminator $D^*(x)$ (fixed G)

For each x , maximize

$$p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x)).$$

Set derivative to 0:

$$\frac{p_{\text{data}}(x)}{D(x)} - \frac{p_g(x)}{1 - D(x)} = 0 \Rightarrow D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

13. Why is D 's objective concave in D ?

For fixed x , it's a weighted sum of $\log D$ and $\log(1 - D)$, both concave on $D \in (0, 1)$. Weighted sum of concave functions is concave.

14. What divergence is minimized at optimum?

With $D = D^*$, optimizing G minimizes **Jensen–Shannon divergence**:

$$\text{JS}(p_{\text{data}} \parallel p_g).$$

● D — Stability / issues

15. Why unstable? Two reasons.

- **Non-convex/non-concave** minimax game $\rightarrow$ oscillations.
- **Vanishing gradients** when D becomes too good.
(Also acceptable: mode collapse, sensitivity to hyperparameters.)

16. If D is too strong early?

$D(G(z)) \approx 0$ so generator gradients become tiny (saturating loss), slowing or stalling G learning.

17. Why does $\min \mathbb{E}_z \log(1 - D(G(z)))$ have poor gradients early?

If $D(G(z)) \approx 0$, then $\log(1 - D(G(z))) \approx \log(1) = 0$ and gradient wrt θ is small ("saturation"), so G gets weak learning signal.

18. Why use $\max \mathbb{E}_z \log D(G(z))$ instead?

If $D(G(z)) \approx 0$, then $\log D(G(z))$ is very negative with **large gradient magnitude**, giving G a stronger learning signal ("non-saturating" generator loss).

● E — Alternating optimization

19. One full iteration of GAN training

1. Sample real batch $x \sim p_{\text{data}}$.

2. Sample noise $z \sim p(z)$, generate fake $G(z)$.
3. **Update D** (fix G): ascend gradient of $\log D(x) + \log(1 - D(G(z)))$.
4. Sample new z .
5. **Update G** (fix D): descend gradient of generator loss (often $-\log D(G(z))$ or $\log(1 - D(G(z)))$).

20. Why not just do standard GD jointly on minimax?

Because it's not a simple minimization problem: simultaneous updates can **chase** each other, causing cycling/instability rather than convergence.

21. Meaning of "choose number of alternating steps carefully"?

You often do **k discriminator updates per 1 generator update**. k too small $\rightarrow D$ weak (bad signal). k too large $\rightarrow D$ too strong (vanishing gradients / instability).

22. If you update D many more times than G ?

D may become near-perfect $\Rightarrow G$ receives weak/unstable gradients; can lead to slow learning or collapse.

F — True/False

23. "Discriminator is trained using gradient descent."

False. In the slide form it's written as a **max** so it's gradient ascent (or equivalently minimize the negative).

24. "At equilibrium, generator distribution equals data distribution."

True. Ideal Nash equilibrium: $p_g = p_{\text{data}}$.

25. "GAN training guarantees convergence to a global optimum."

False. No general guarantee; can oscillate or diverge.

26. "Cross-entropy encourages calibrated probabilities."

True (in principle). As NLL, it is a proper scoring rule; in practice deep nets may still be miscalibrated without regularization/temperature scaling.

G — Oral / explanation

27. Explain GAN as a game

A two-player zero-sum game: D tries to maximize correct classification; G tries to minimize it by producing samples indistinguishable from real ones. Training seeks a Nash equilibrium.

28. Why can GANs produce sharper images than VAEs?

GANs use an adversarial loss that can better match perceptual/realistic high-frequency details, while VAEs often optimize likelihood bounds with pixel-wise terms that can average modes $\rightarrow$ blur.

29. Meaning of "G fools D"?

Generated samples $G(z)$ are so realistic that $D(G(z))$ is close to 1 (or at least near 0.5 at equilibrium), meaning D can't reliably tell fake from real.

30. What is mode collapse? Relation to D loss?

Mode collapse: G produces low-diversity outputs (few modes) that still trick D . D loss might not immediately reveal missing modes because G exploits weaknesses in D rather than matching full data distribution.

★ Professor-favorite (the one I highlighted earlier)

If D is optimal at every step, what is G minimizing?

With $D = D^*$, the GAN objective reduces to minimizing

$$JS(p_{\text{data}} \parallel p_g)$$

(up to constants). Equilibrium when $p_g = p_{\text{data}}$.

If you want, I can turn this into a **one-page cheat sheet** (definitions + objectives + key results like D^* and JS divergence) you can memorize fast.



